ShakTalk – Call Latency & Dropped Speech: Fixes and Server Plan

Oct 20, 2026 · @shak

Summary

The 3–6 second delays and dropped speech come from how the app handles audio, not from Gemini being slow. Six code fixes are ready for review; the seventh (server location) is an infrastructure change for the developer.

- **Branch:** fix/latency-and-dropped-speech
- **Pull request:** PR #2, merging into `feature/translation-updates`
- **Files changed:** `backend/geminiService.js`, `backend/server.js`, `src/screens/SessionScreen.tsx`, `src/hooks/useAudioRecorder.ts`, `src/contexts/WebSocketContext.tsx`, `src/hooks/useWebSocket.ts`

Expectation: the app translates after each sentence ends. After these fixes the listener should wait roughly the sentence length plus 0.7–1.2 seconds, without the extra multi-second stalls or lost words. It will not behave like simultaneous interpretation.

Code fixes in PR #2

Each row is one root cause, the symptom it caused, and what changed.

#	Problem	Symptom	File	Fix
1	Phone cleared the playlist when a new turn's first audio arrived	Previous sentence cut off mid-way	`SessionScreen.tsx`	Playlist resets only after all queued audio has played; audio pieces are written and queued in order
2	Gemini's default "interruption" mode cancelled its translation when the speaker kept talking after a short pause	Phrases dropped	`geminiService.js`	Set `activityHandling: 'NO_INTERRUPTION'`

#	Problem	Symptom	File	Fix
3	Mic chunks were dropped entirely while the partner's audio played	Your words lost during and just after playback	`SessionScreen.tsx`	Gated chunks are replaced with silence (echo still blocked)
4	Because nothing was sent, Gemini never heard the speaker stop	3–6 second stalls before translation	`SessionScreen.tsx`	Same silence fix lets Gemini end the turn on time; mute flush raised to 800 ms
5	Server resent the whole turn's audio at turn end (~65 KB per second of speech)	Next turn's audio delayed on mobile data	`server.js`	Final message now carries text only; recordings still uploaded
6	No Gemini session lifetime handling	Mid-call stalls, lost words, error pop-ups on reconnect	`geminiService.js`	Session resumption, context compression, connection swap on `goAway`, instant auto-reconnect, ordered ~3 s buffer during reconnects

Smaller fixes in the same PR:

- Faster audio decoding in the mic hook and far fewer screen re-renders during a call.
- `turnId` is now passed to the transcript callbacks, so duplicate transcript lines are filtered as intended.

Verification done: TypeScript check clean (0 errors), backend syntax check passes, and the reconnect logic passed 18 of 18 scenarios against a simulated Gemini Live connection.

Testing before merge

Run a real two-phone call before merging; the simulated tests can't cover echo or real network conditions.

- [ ] Deploy the backend from the branch and install a fresh app build on two phones.
- [ ] Loudspeaker on both phones: confirm the translation is not picked up and re-translated (no echo loop).

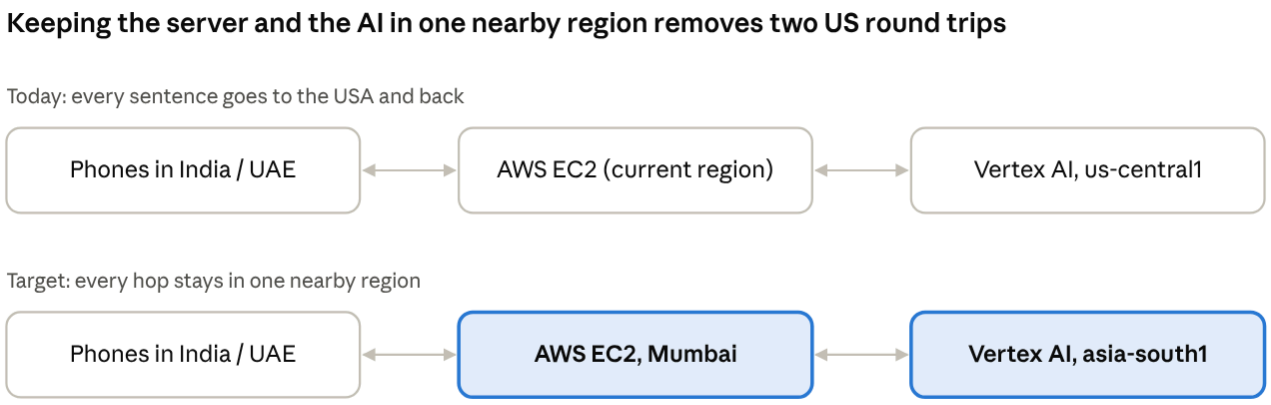
- [ ] Speak long sentences with natural pauses: confirm no part of the translation is cut off.
- [ ] Reply immediately after hearing a translation: confirm your first words are translated.
- [ ] Tap mute mid-sentence: confirm what you already said is still delivered.
- [ ] Stay on one call for more than 15 minutes: confirm translation keeps working without an error pop-up.

Server log lines to watch:

- `[LATENCY][server] ... first Gemini audio chunk, Xms since last input chunk` shows how long Gemini takes per turn. Use it as the baseline for the server-location change.
- `goAway received ... swapping connection` and `Session ready ... gen=N` show a connection swap working.
- `Flushing N queued chunks` shows audio held during a reconnect and then sent.

Server location (fix 7, for the developer)

Moving the backend and Google's AI region close to each other and to the users should cut a few hundred milliseconds from every sentence. Today each sentence travels from the phone in India or the UAE to the AWS server, then to Vertex AI in `us-central1` (USA), and back, which is two long round trips.



call audio route · today vs target

The target row assumes Mumbai is available for the model; if not, the same idea applies with a European region for both.

1. **Measure the baseline.** Note which AWS region the EC2 server runs in. Run a few test calls and record the `[LATENCY][server] ... first Gemini audio chunk` numbers.
2. **Pick the nearest Google region that serves the model.** Google's locations page lists `gemini-live-2.5-flash-native-audio` in many US and European regions plus a `global` endpoint. Check its Asia Pacific rows for Mumbai (`asia-south1`) and

Singapore (`asia-southeast1`). A Google forum thread reported an earlier preview worked only in `us-central1`, so confirm with a real call.

3. **Switch the AI region.** Set the server environment variable `GOOGLE_CLOUD_LOCATION` to the chosen region, in this order of preference: `asia-south1` if available, otherwise a European region such as `europe-west4`, or `global`. Restart and compare the latency numbers against the baseline. No code change is needed.
4. **Move the EC2 server next to it.** Use AWS Mumbai (`ap-south-1`) when the AI runs in Mumbai, or a nearby AWS EU region when it runs in Europe. Then point the DNS for `shaktalk.com` at the new server.
5. **Re-test** with the checklist above and keep whichever region gives the lowest numbers.

Warning: call sessions are stored in the server's memory, so both phones on a call must reach the same server instance. A single EC2 instance does this correctly. Do not move to auto-scaling Cloud Run or several instances without a single instance or sticky routing, or calls will fail with "session not found".

Other follow-ups

- [ ] **Sync the lockfile.** `package-lock.json` is out of date with `package.json` (`expo-updates`, `expo-build-properties` were added), so `npm ci` fails. Run `npm install` once and commit the updated lockfile.
- [ ] **Survive network blips.** If either phone's WebSocket drops briefly, the server ends the whole call. Add client reconnect plus a short grace period on the server before cleanup.
- [ ] **Optional model test.** Google's docs now list a newer `gemini-3.8-live` model. The server already reads `GEMINI_LIVE_MODEL`, so one test call with that variable set compares speed and quality without a code change.
- [ ] **Merge to master.** `master` is about 30 commits behind `feature/translation-updates`; merge once this PR is tested.

Sources

- Vertex AI generative AI locations
- Gemini 2.5 Flash with Gemini Live API
- Google Developer forum: Live native-audio region availability